

Lab 02 — Elliptic curves, ECDSA, HD wallets and post-quantum cost

Khusanjon Ashuraliev · Introduction to Blockchain Programming · Óbuda University, NIK · October 2026

Item	Value
Repository	https://github.com/KhusanAshuraliev/lab-02-keys-and-crypto
Stack	Python 3.11+ with uv. Standard library only, plus pycryptodome for Keccak-256. No ecdsa, coincurve, eth-keys or bip_utils.
Evidence	Output of every demonstration and both test runs: evidence/ in the repository
Reproduce	<code>uv sync && uv run pytest && uv run python -m crypto_lab</code>
Tests	113 passed: the course suite of 64 tests, copied unchanged, plus 49 own tests.

I copied all output below from a run of my own implementation (evidence/demo_*.txt). I kept the module and function names of the course suite so that it runs unchanged; the code is my own.

1. Part A – curve arithmetic

Group-law verification output

```
p = 0xffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff
n = 0xfffffffffffffffffffffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141
G = Point(0x79be667ef9dcbac55a06295ce870b07029bfcd2dce28d959f2815b16f81798, 0x483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8)
G on curve : True

Group law checks:
  n*G is the identity           : True
  (n-1)*G + G is the identity  : True
  G + G == 2*G                 : True
  double-and-add == Montgomery ladder : True    (20 random scalars)

Edge cases:
  G + 0 == G                   : True
  G + (-G) is the identity     : True
  (n-1)*G == -G               : True

2G:
  x = 0xc6047f9441ed7d6d3045406e95c07cd85c778e4b8cef3ca7abac09b95c709ee5
  y = 0x1ae168fea63dc339a3c58419466ceaeef7f632653266d0e1236431a950cfe52a

SEC1 serialisation of G:
  compressed (33 B) : 0279be667ef9dcbac55a06295ce870b07029bfcd2dce28d959f2815b16f81798
  uncompressed (65 B) : 0479be667ef9dcbac55a06295ce870b07029bfcd2dce28d959f2815b16f81798483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199c47d08ffb10d4b8
  parse(serialise(G)) == G, both forms : True
```

Check	Result	README expects
G on curve and the four group-law checks	all True	all True ✓
x-coordinate of 2G	0xc6047f94...5c709ee5	published value ✓
Course test suite	64 passed	64 passed ✓

Why $n \cdot G = O$ must hold

The points of `secp256k1` form a finite group under addition, and `G` generates a cyclic subgroup of it. Adding `G` to itself repeatedly cannot produce new points forever, so the sequence `G`, `2G`, `3G`, ... must return to the

identity O . The smallest positive n with $n \cdot G = O$ is the **order** of G . For secp256k1 the whole curve has prime order n , so every point other than O has exactly this order.

The consequence is that scalars only matter modulo n : $(k + n) \cdot G = k \cdot G + n \cdot G = k \cdot G$. Private keys, nonces and signatures therefore live in $\mathbb{Z}_n$, and every scalar computation in ECDSA is reduced mod n . The second check says the same thing from the other side: $(n-1) \cdot G$ is the inverse of G , so adding G gives O .

My `scalar_mul` does not reduce the scalar first, so `scalar_mul(n, G)` really runs the full Double-and-Add over the 256 bits of n and lands on O only because addition, doubling and the modular inverse are all correct. That makes the check a real test of the arithmetic.

What would break if the implemented order were wrong by one

Suppose I had implemented $n' = n \pm 1$ in place of n . Then $n' \cdot G = \pm G$, not O , and reduction mod n' no longer matches what the curve does. Everything that reduces mod the order uses the wrong modulus:

- the nonce inverse k^{-1} is computed mod n' (and $n' = n + 1$ is even, so it is not even prime: Fermat's inverse stops being an inverse);
- $s = k^{-1}(z + r \cdot d)$ and the verifier's $u_1 = z \cdot s^{-1}$, $u_2 = r \cdot s^{-1}$ are reduced mod n' , so $u_1 \cdot G + u_2 \cdot Q$ no longer equals $k \cdot G$ and **valid signatures stop verifying**;
- the range checks on keys, nonces, r and s accept or reject the wrong values: with $n + 1$ the scalar n passes as a "valid" private key although its public key is the point at infinity;
- low- s normalisation $n' - s$ gives a value that is not the negation of s , so the flipped signature is invalid, and BIP-32 child keys $(I_L + k_{par}) \bmod n'$ no longer match the public derivation $I_L \cdot G + K_{par}$.

Constant-time variant

`scalar_mul_ladder` is a Montgomery ladder: it always runs 256 iterations and each performs exactly one addition and one doubling, whatever the bit. I checked that it agrees with Double-and-Add on 20 random scalars in the demo and on 30+ scalars in my tests (including 1, $n-1$, n and $2^{256}-1$). Its limits are discussed in section 5.

2. Part B — ECDSA and nonce reuse

```
private key d = 0xc0ffee00badc0de1234567890abcdef1122334455667788990aabbccddeeff00
address       = 0xa0f598084f711360A38D861F8Fb2100949170C09

Signature with an RFC 6979 nonce:
r = 0x85760d15466a56955362c0aec3f4a58d3013f16bdf18de581cc2bc383c55744f
s = 0x6c0877227536cd11c646e2f6457183bc8b1534f466a68dd73347d43c98882951
recovery id = 0
verifies                    : True
reproducible across runs   : True
rejects a different message : True
low-s enforced (s <= n/2)  : True

Public-key recovery:
recovered key matches signer : True

Malleability:
(r, n-s) also verifies      : True

Nonce reuse -- two messages signed with the same k:
signature 1 r = 0x41eb18ff5ae68f6cb2f3c4000767e88e003f2e4af2193d3c973db291b16cec56
            s = 0x4f31c38dc63ba7d0c1302247f0bba175951f0e86108f8cbbf4fb1e8f654f91ce
signature 2 r = 0x41eb18ff5ae68f6cb2f3c4000767e88e003f2e4af2193d3c973db291b16cec56
            s = 0x5db4bccebdbc5d1a11cb889c60677205407db7e951f348b2a4f8b6c3cf4143de
identical r      : True
original d       = 0xc0ffee00badc0de1234567890abcdef1122334455667788990aabbccddeeff00
recovered d      = 0xc0ffee00badc0de1234567890abcdef1122334455667788990aabbccddeeff00
attack succeeded : True

The same attack for every low-s sign combination:
neither s flipped      : True
first s flipped        : True
second s flipped       : True
both s flipped         : True
```

In my run every line of the README's expected table is True, the address is 0xa0f598084f711360A38D861F8Fb2100949170C09, and the **recovered d equals the original d exactly**. As messages I used SHA-256 of "Lab 02: pay 1 ETH to Alice" and "Lab 02: pay 2 ETH to Bob".

The attack

Both signatures use the same k, so both have the same $R = k \cdot G$ and the same r. Subtracting the two signing equations removes the unknown d:

$$s_1 - s_2 = k^{-1}(z_1 - z_2) \implies k = (z_1 - z_2) / (s_1 - s_2), \quad d = (s_1 \cdot k - z_1) / r \pmod{n}$$

The attacker needs only the two signatures and the two message hashes, and the reuse is visible to anyone because the r values are equal.

Low-s. Real signatures are normalised, so either s may have been replaced by $n - s$, which is the signature made with $-k$. Flipping both only negates k and leaves d unchanged, so two cases remain: the signs agree (denominator $s_1 - s_2$) or differ ($s_1 + s_2$). My function computes d for both and returns the candidate whose public key $d \cdot G$ verifies both signatures. It returns the exact key in all four combinations, as shown above and asserted in test_extra.py.

Why RFC 6979 removes the vulnerability without making k predictable

RFC 6979 derives the nonce as $k = \text{HMAC-DRBG}(d, z)$: a deterministic function of the private key and the message hash.

- **k never repeats across messages.** A different message gives a different z and therefore a different k, so two different messages never share r under one key. Signing the same message twice gives the same signature, which reveals nothing new. The attack needs two *different* messages with one k, and that can no longer happen.

- **k is still secret.** The private key *d* is the HMAC key. Someone who does not know *d* cannot compute *k*, and cannot distinguish it from a random value. Determinism only removes the dependence on a random number generator, which is what failed in the PlayStation 3 case (a fixed nonce) and in the 2013 Android wallets (a broken SecureRandom). It does not make *k* public.

My implementation reproduces the widely published vector for key 1 and message "Satoshi Nakamoto" (*k* = 0x8f8a276c...74f15d15), so the derivation agrees with other implementations.

3. Part C — HD wallets

```
mnemonic   : abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon abandon about
seed       : 5eb00bbddcf069084889a8ab9155568165f5c453ccb85e70811aaed6f6da5fc19a5ac40b389cd370d086206dec8aa6c43da
ea6690f20ad3d8d48b2d2ce9e38e4
master key : 0x1837c1be8e2995ec11cda2b066151be2cfb48adf9e47b151d46adab3a21cdf67
chain code : 7923408dadd3c7b56eed15567707ae5e5dca089de972e07f3b860450e2a3b70e

Accounts:
m/44'/60'/0'/0/0 -> 0x9858EfFD232B4033E47d90003D41EC34EcaEda94
m/44'/60'/0'/0/1 -> 0x6Fac4D18c912343BF86fa7049364Dd4E424Ab9C0
m/44'/60'/0'/0/2 -> 0xb6716976A3ebe8D39aCEB04372f22Ff8e6802D7A

Same mnemonic, different passphrase (address 0):
passphrase ''      -> 0x9858EfFD232B4033E47d90003D41EC34EcaEda94
passphrase 'correct horse' -> 0xFf0Bdf99513306994d4aF9949BA7cfA60204D285
passphrase 'correct horse ' -> 0x67dA015b6A80F19822e5d270A7788F72f018a724

Parent-key recovery from the xpub of m/44'/60'/0'/0 and child key /0:
xpub holds a private key      : False
recovered parent key          : 0xa29ac2cb17e31cdab42a8fe2d83f04f4b69c5e73bc8d3bf6b5dc96ac239b145a
matches the real parent       : True
sibling /1 key now derivable  : True
same attack on a hardened child : refused -- hardened children are immune: I_L depends on the parent private key

Watch-only derivation:
xpub-derived address matches : True
hardened child from an xpub   : refused -- hardened derivation requires the parent private key

Addresses:
keccak256(b'') = c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470
sha3_256(b'') = a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a (a different function)
EIP-55 0x5aAeb6053F3E94C9b9A09f33669435E7Ef1BeAed matches the EIP : True
EIP-55 0xfB6916095ca1df60bB79Ce92cE3Ea74c37c5d359 matches the EIP : True
EIP-55 0xdbF03B407c01E7cD3CBea99509d93f8DDDC8C6FB matches the EIP : True
EIP-55 0xD1220A0cf47c7B9Be7A2E6BA89F429762e7b9aDb matches the EIP : True
```

Value	Result	Expected
seed	5eb00bbddcf06908...f6da5fc1...	BIP-39 vector ✓
m/44'/60'/0'/0/0	0x9858EfFD232B4033E47d90003D41EC34EcaEda94	✓
m/44'/60'/0'/0/1	0x6Fac4D18c912343BF86fa7049364Dd4E424Ab9C0	✓
m/44'/60'/0'/0/2	0xb6716976A3ebe8D39aCEB04372f22Ff8e6802D7A	✓
passphrase 'correct horse'	0xFf0Bdf99513306994d4aF9949BA7cfA60204D285	✓
passphrase 'correct horse '	0x67dA015b6A80F19822e5d270A7788F72f018a724	✓ unrelated
parent-key recovery	matches the real parent : True	✓
watch-only derivation	xpub-derived address matches : True	✓

The passphrase demonstration

In BIP-39 the mnemonic is the PBKDF2 *password* and the passphrase is appended to the *salt* ("mnemonic" + passphrase). There is no stored value to compare a passphrase against, so there is no wrong passphrase: each one produces a valid 512-bit seed and a valid wallet. 'correct horse' and 'correct horse ' differ by

one trailing space and give unrelated addresses. This allows plausible-deniability wallets, and it also means a mistyped passphrase shows an empty wallet with no error.

The parent-key recovery attack

For a non-hardened child with index i , BIP-32 computes

$$I = \text{HMAC-SHA512}(c_{\text{par}}, \text{serP}(K_{\text{par}}) \parallel i), \quad k_{\text{child}} = (I_L + k_{\text{par}}) \bmod n \implies k_{\text{par}} = (k_{\text{child}} - I_L) \bmod n$$

The chain code c_{par} and the public key K_{par} are exactly the contents of the xpub, so anyone holding the xpub can compute I_L . One leaked child private key then gives the parent private key by a single subtraction. In the demo I gave the function only the neutered node of `m/44'/60'/0'/0` and the private key of child 0; it returned `0xa29ac2cb...239b145a`, the real parent key, and with it I derived the private key of sibling 1. The whole branch is lost, not one address.

Why the same attack fails against a hardened child

For a hardened child (index $\geq 2^{31}$) the HMAC input changes to

$$I = \text{HMAC-SHA512}(c_{\text{par}}, 0x00 \parallel \text{ser256}(k_{\text{par}}) \parallel i)$$

I_L now depends on the parent **private** key. To undo $k_{\text{child}} = I_L + k_{\text{par}}$ the attacker would first have to compute I_L , which requires k_{par} , the value being sought. The xpub contains nothing that helps. My function **refuses** a hardened index with a `ValueError`, and my tests assert that. A further test shows that the non-hardened formula applied to a hardened child gives a wrong key. For the same reason an xpub cannot derive hardened children at all, which the last line of the watch-only block shows.

This is why BIP-44 hardens `purpose'`, `coin_type'` and `account'`: a leak below an account cannot climb above it.

The real deployment scenario

A merchant, payment processor or exchange puts an **account xpub on a web server** so that it can generate a fresh deposit address per customer without holding spending keys. Separately, one child private key leaks, for example from a hot wallet that signs withdrawals, a compromised device, or a key exported into another application. The two pieces together give the account's private key and every address under it, including all future ones. The xpub is meant to be safe to publish, so its exposure is not treated as an incident; that is what makes the combination dangerous.

4. Part D — post-quantum cost

Scheme	PQ	pk (B)	sig (B)	tx (B)	× ECDSA	tps
ECDSA (secp256k1)	no	33	72	215	1.0×	7.75
Schnorr (BIP-340)	no	32	64	206	1.0×	8.09
ML-DSA-44	yes	1312	2420	3842	17.9×	0.43
ML-DSA-65	yes	1952	3309	5371	25.0×	0.31
SLH-DSA-128s	yes	32	7856	7998	37.2×	0.21
FN-DSA-512	yes	897	666	1673	7.8×	1.00

Model: one signed input, one output, 110 bytes of non-signature overhead, 1 MB block every 10 minutes. $\text{tx} = 110 + \text{pk} + \text{sig}$; $\text{tps} = 1\,000\,000 / \text{tx} / 600$. Output of `python -m crypto_lab pqc`; identical to the README's table.

Category	Status	Reason
p2pkh_unspent	PROTECTED (for now)	Only HASH160(pubkey) is on chain. Shor needs the public key; Grover only square-roots a hash search. Protection ends when the output is spent.
p2pk	EXPOSED	The public key itself is the output script. Includes the earliest mining rewards.
reused_address	EXPOSED	An earlier spend already published the key, and funds remain behind it.
mempool_transaction	EXPOSED (briefly)	Between broadcast and confirmation the key is public while the coin is unspent.
ethereum_any_active	EXPOSED	ecrecover gives the public key from any signature, so every account that has ever sent a transaction is exposed.

What the table says

Shor's algorithm does not weaken ECDSA and Schnorr; it solves the discrete logarithm, so both fail completely. Replacements exist and are standardised, so the obstacle is size. A transaction grows by a factor of 8 to 37, and on a fixed block size throughput falls in the same proportion, from 7.75 tps to between 1.00 and 0.21. FN-DSA is the smallest but needs floating-point Gaussian sampling, which is hard to implement safely; SLH-DSA rests only on hash functions but has the largest signatures; ML-DSA is the middle option.

An asset that must stay secure for fifteen years: decide now or later?

Now. My argument does not depend on when a quantum computer will exist.

A blockchain is public and permanent. Every public key that is on chain today (P2PK outputs, reused addresses, every Ethereum account that has transacted) can be copied today and kept. This is harvest now, decrypt later: the attacker does not need the machine at the time of collection, only at some point while the asset is still worth taking. For signatures the "decryption" is computing the private key from the stored public key and spending the coins.

So I look at two durations that we do know. The asset must stay secure for $X = 15$ years. Migration takes Y years: choosing a scheme, changing consensus rules, upgrading wallets and hardware, and getting every holder to move coins to new outputs, which on a decentralised chain is slow and never complete. If a capable machine appears after Z years, the asset is safe only if $X + Y < Z$. We do not know Z , but with $X = 15$ and Y of several years, the condition already requires that no such machine exists for about two decades. Betting an asset on that is a decision in itself. And exposure cannot be undone later: a key that is public today stays public.

The costs in the table make waiting more dangerous, not less: because post-quantum signatures cut throughput by an order of magnitude, the migration cannot be rushed through in a few blocks when a threat becomes concrete. In the meantime the cheap measure is to keep keys behind hashes: never reuse an address, and move long-term holdings to outputs whose public key has never been published.

5. Implementation risk

A signature that verifies proves only that (r, s) is consistent with the public key. It says nothing about how k and d were handled on the way. Three ways the key can leak while every signature still verifies:

1. Timing and power side channels. In Double-and-Add the point addition runs only for the 1-bits of the scalar, so run time grows with the Hamming weight of k and a power trace shows the bits directly. Knowing k for one signature gives $d = (s \cdot k - z)/r$, and even a few leaked bits per signature are enough (see 2). Mitigation: constant-time field arithmetic, a ladder with no secret-dependent branch or memory access, scalar blinding.

My code has this problem. `ecdsa._sign_with_nonce` calls `curve.scalar_mul(k, G)`, which branches on every bit of `k`, and computes `k-1` with `inverse_mod(pow)`, whose time depends on its operand. `scalar_mul_ladder` equalises the sequence of point operations but still branches on the bit, and Python's big integers are variable-time, so it does not fix it either. HD derivation leaks the same way through `private_to_public`.

2. Bad nonces. A repeated `k` gives the same `r`, and `d` follows from two signatures (Part B). A `k` from a weak RNG can be guessed. A `k` that is merely biased, for example a few top bits always zero, repeats no `r`, yet a lattice attack on the Hidden Number Problem recovers `d` from a few hundred signatures. Mitigation: RFC 6979, and never a caller-supplied nonce. In my code `ecdsa.sign` accepts a nonce argument for the demonstration; any caller using it reintroduces the bug, so a real API would have to remove it.

3. Faults. If a bit flips during signing (voltage glitch, Rowhammer, bad hardware) the result is a faulty signature. With deterministic nonces, a correct and a faulty signature of the same message share `k` but differ elsewhere, and `d` can be solved from the pair. Mitigation: verify every signature before releasing it, and add fresh randomness to the nonce derivation. My `sign` returns the signature without verifying it first.

A fourth, mundane channel is `d` or `k` reaching logs, exception messages, swap or core dumps. My CLI prints the demo key on purpose, which is acceptable only because it is a published test key.

(379 words)